



Il way-stack. La mappa completa.

Tutto quello che hai installato con 3 comandi: cosa c'è dentro, a cosa serve ogni pezzo, quando usarlo. Una pagina per pezzo, in parole semplici.

- 01 Cos'è il way-stack
- 02 Dove si trova — il repo GitHub
- 03 Vocabolario — plugin, skill, comando
- 04 Le fondamenta — orchestrator, vault, hook
- 05 I 6 plugin (+ caveman)
- 06 La agent factory — i 5 comandi
- 07 Le 7 skill incluse
- 08 Memoria + grafo — claude-mem e graphify
- 09 Design — frontend-design + SHIN-EN + 6 design skills
- 10 Gli extra — agent monitor e deploy agent
- 11 I 3 framework opzionali — GSD, BMAD, gstack
- 12 I documenti di riferimento — SDD, Mega Prompt, template
- 13 Cheat-sheet — tutti i comandi

Cos'è il way-stack

Il way-stack è **il mio setup completo di Claude Code**, impacchettato e pubblicato gratis su GitHub. Dentro c'è tutto quello che uso ogni giorno per lavorare con gli agent: il cervello che decide quale strumento usare, la libreria dove vive la conoscenza, i plugin che danno capacità in più, e la catena di montaggio che costruisce agent nuovi.

Senza stack, ogni volta dovresti spiegare a Claude come lavorare. Con lo stack, Claude **sa già il metodo**: fa le domande prima di scrivere codice, salva quello che impara, controlla il lavoro prima di consegnarlo.

In una frase: due anni del mio lavoro di configurazione, tre comandi per averlo identico sul tuo computer.

Dove si trova

Il way-stack è pubblico e gratuito su GitHub. Lì trovi il codice, il README con le istruzioni di installazione sempre aggiornate, e gli update futuri.

Repo: github.com/Lucioway/way-stack — l'installazione la facciamo insieme nella lezione di setup.

Le 3 parole da sapere

PAROLA · 01

Plugin

Un pacchetto di capacità in più che installi dentro Claude. Come un'app sul telefono: la metti, e Claude sa fare cose nuove.

PAROLA · 02

Skill

Una ricetta che Claude sa seguire passo passo. Tu chiedi, lui usa il metodo giusto senza che glielo spieghi ogni volta.

PAROLA · 03

Comando

Una scorciatoia che inizia con `/`. Scrivi `/agent-spec` e parte tutta la procedura. Un tasto, un lavoro.

Le fondamenta — orchestrator, vault, hook

CERVELLO

Orchestrator
Il CLAUDE.md principale: il vigile che smista. Legge la tua richiesta e decide da solo quale skill o plugin usare. Tu chiedi, lui instrada.

LIBRERIA

Vault PARA
La cartella dove vive tutta la conoscenza: progetti, decisioni, appunti, materiale. Organizzata sempre allo stesso modo, così Claude trova tutto.

DIARIO

Hook di sessione
Il diario automatico. A fine sessione scrive da solo cosa è stato fatto. Tu non scrivi niente: la cronologia si costruisce da sola.

Insieme: il vigile decide, la libreria ricorda, il diario registra. Questa è la base su cui gira tutto il resto.

I 6 plugin

PLUGIN	COSA FA, IN PAROLE SEMPLICI	QUANDO LO USI
superpowers	Dà a Claude un metodo di lavoro: prima le domande, poi il piano, poi il build, poi il controllo. Senza, parte a scrivere codice a caso.	Sempre. Si attiva da solo.
frontend-design	Fa uscire pagine, slide e PDF belli — niente grafica "da AI", direzione precisa. Lavora con la grafica SHIN-EN.	Ogni volta che serve qualcosa di visivo.
code-review	Fa controllare il codice da più revisori AI insieme: uno cerca i bug, uno la sicurezza, uno le cose scritte male.	Prima di usare codice importante.
cli-anything	Comanda da terminale i programmi con interfaccia grafica (GIMP, Blender, LibreOffice...). L'agent li usa senza aprire finestre.	Quando un agent deve usare un programma "da click".
ralph-loop	Fa ripetere il lavoro a Claude da solo, un pezzo alla volta, finché la lista non è finita.	Lavori lunghi: "sistemami tutte e 100 le pagine".
claude-mem	La memoria automatica: a ogni sessione salva da solo cosa è successo e te lo ripresenta la volta dopo. Claude "si ricorda" senza che tu scriva niente.	Sempre. Lavora in sottofondo.

Bonus — caveman: un add-on che rende Claude telegrafico: via i giri di parole, restano solo i fatti. Risposte più corte = meno token = meno costi. Si accende e si spegne quando vuoi.

La agent factory — la catena di montaggio

Cinque comandi in fila: entra un'idea, esce un agent finito. È il pezzo che usi per costruire qualsiasi agent nuovo.

- 1 **/agent-spec**
Scrivi il contratto: cosa entra, cosa esce, con quali regole. In italiano semplice. Mai codice prima della spec.
- 2 **/agent-tasks**
Trasforma la spec in una lista di lavori numerati. La to-do list dell'agent, scritta da lui.
- 3 **/agent-execute**
Esegue i lavori uno alla volta. Ogni lavoro finito = un salvataggio: se qualcosa va storto, torni indietro di un passo.
- 4 **/agent-verify**
Controlla che ogni punto della spec sia coperto davvero. Mai fidarsi del primo run.
- 5 **/agent-ship**
Consegna: l'agent è pronto da usare, documentato e verificato.

Regola d'oro: spec all'inizio, verify alla fine. È un cerchio che si chiude — tutto quello che sta in mezzo lo fa l'agent.

Le 7 skill incluse

Le ricette che arrivano dentro il way-stack. Le prime tre sono **il bibliotecario** della tua libreria, le altre quattro sono operai specializzati.

SKILL	COSA FA, IN PAROLE SEMPLICI
vault-ingest	Mette dentro: prende un file grezzo (transcript, PDF, appunti), lo spezza in pagine ordinate, dà un titolo a ognuna e aggiorna l'indice.
vault-query	Trova: cerca nel vault e ti risponde. "Cosa avevo deciso sul pricing?" — pesca la pagina giusta e sintetizza.
vault-lint	Pulisce: controlla la salute della libreria — pagine orfane, contraddizioni, roba vecchia. Segnala il disordine.
create-agent	Il capofficina: quando dici "fammi un agent che...", lui prende in mano la richiesta e la porta dentro la agent factory.
deploy-project	Mette online: pipeline di 10 passi per pubblicare un progetto sul web (controllo sicurezza in 5 livelli, test, anteprima, produzione).
shinen-design	La mia grafica SHIN-EN per le dashboard: nero giapponese minimale, un solo foglio di stile, numeri giganti in trasparenza. I tuoi tool escono tutti con la stessa faccia.
handoff	Il passaggio di consegne: chiudi una sessione lunga e lui prepara il riassunto per la prossima — riparti senza rispiegare niente.

La memoria — claude-mem e graphify

Due pezzi che lavorano insieme. **claude-mem** (plugin) è il diario automatico: salva da solo cosa succede in ogni sessione e lo ripresenta la volta dopo — Claude si ricorda di te e del lavoro fatto. **graphify** è la mappa: con `/graphify` trasforma una cartella di file in un grafo — i concetti diventano punti, i collegamenti diventano linee.

Una volta costruito, il grafo si interroga direttamente da Claude: "quali sono i concetti più collegati?", "che strada c'è tra retention e bundle?". Risposta in secondi, senza cercare a mano.

In breve: claude-mem = il diario (si ricorda). Graphify = la mappa (collega). Servono entrambi — li vediamo all'opera nella lezione sulla memoria.

Frontend-design + SHIN-EN + 6 design skills

Il lato visivo dello stack. **Frontend-design** è il plugin che impagina; **SHIN-EN** è la mia grafica — bianco e nero, scritte grandi, zero decorazioni inutili. La vedi in queste pagine, nelle slide del corso, nei miei PDF. È nel way-stack: **la usi anche tu, gratis**.

Insieme arrivano **6 design skills**: regole di design professionale che si attivano da sole quando Claude tocca qualcosa di grafico — **refactoring-ui** (sistema gerarchia, spazi e colori), **ux-heuristics** (controlla l'usabilità), **hooked-ux** (loop di ritorno e retention), **design-sprint** (validare un'idea in 5 giorni), **ios-hig-design** (app iPhone fatte bene), **ui-ux-pro-max** (design system completi per SaaS e dashboard).

Il pattern: Claude scrive la pagina HTML in SHIN-EN → Chrome headless (Chrome senza finestra) la stampa in PDF. Report, guide, preventivi: stesso giro per tutto.

Gli extra — agent monitor e deploy agent

Agent monitor è la sala di controllo: una dashboard 3D in tempo reale che mostra tutta la tua flotta di agent in una pagina sola. Vedi chi sta girando, chi ha finito, chi è in errore, e gli ultimi output di ognuno. Quando hai un agent solo non serve; quando ne hai cinque che lavorano di notte, è l'unico modo per non perdere il filo. Arriva come template: lo accendi quando la flotta cresce.

Deploy agent è l'addetto alla pubblicazione, la versione "da terminale" della skill deploy-project. Il problema che risolve: mettere online un progetto a mano significa dimenticarsi sempre qualcosa — una password lasciata nel codice, un file che non doveva uscire, un test saltato. Il deploy agent fa la trafila completa da solo: **scansiona il progetto** (capisce cos'è e come va pubblicato) → **audit di sicurezza** (cerca chiavi, password e dati sensibili prima che finiscano online) → **test** (verifica che funzioni) → **pubblicazione** (lo mette sul web e ti dà il link). Un comando, progetto online, senza brutte sorprese.

Differenza: deploy-project è la skill che usi dentro Claude ("pubblica questo progetto"); il deploy agent è il template per farne un agent autonomo tuo, che pubblica da solo quando glielo chiedi.

I 3 framework — GSD, BMAD, gstack

Un framework è un **metodo di lavoro completo** per i progetti grandi: non una skill singola, ma un percorso con fasi, controlli e documenti. Il bootstrap ti chiede se installarli — sono opzionali perché per i lavori piccoli bastano i plugin. Quando il progetto cresce, ne scegli **uno solo** (mai mischiarli) in base a come ti piace lavorare.

1 GSD — il metodo del solista organizzato (~70 skill)

Per chi lavora da solo su progetti di settimane. Spezza il progetto in fasi e ogni fase segue sempre lo stesso giro:

discuti

(capisce cosa vuoi davvero) →

pianifica

(scrive il piano dei lavori) →

esegui

(un task = un salvataggio, sempre reversibile) →

verifica

(controlla il risultato contro l'obiettivo, non solo "ho finito i task"). Ogni fase parte con la memoria pulita, così Claude non si confonde nei progetti lunghi. Ha anche la modalità notturna:

`/gsd-autonomous` esegue tutte le fasi rimanenti mentre dormi.

2 BMAD — il metodo agile coi documenti (v6, 21 ruoli)

Per chi vuole lavorare come una squadra di sviluppo professionale. Claude interpreta i ruoli uno alla volta: l'

analista

scrive il brief del prodotto, il

product manager

i requisiti, l'

architetto

il disegno tecnico, lo

scrum master

spezza tutto in storie piccole, lo

sviluppatore

le costruisce una alla volta. La regola: i documenti comandano — prima si scrive cosa fare, poi si fa. Perfetto se ti piace avere tutto nero su bianco prima di partire.

3 gstack — l'azienda virtuale completa (~38 skill)

Simula un'intera azienda di software: c'è il
CEO

che giudica l'idea (`/office-hours` per il brainstorming), gli
ingegneri

che rivedono il piano, il
designer

che controlla la grafica, il
QA

che testa, il responsabile
sicurezza

(`/cso`) che fa l'audit, e alla fine `/ship` per consegnare. Ogni decisione passa da più
"persone" con punti di vista diversi. Per prodotti completi dove vuoi il massimo dei controlli.

Quale scegliere: lavoro piccolo → nessun framework, bastano i plugin. Progetto strutturato da solista → **GSD**. Ti piacciono documenti e metodo agile → **BMAD**. Prodotto completo col massimo dei controlli → **gstack**. E ricorda: l'orchestrator conosce tutti e tre e ti propone quello giusto da solo.

I documenti di riferimento

Nel repo c'è anche una cartella di **documenti di metodo**: non si installano, si leggono — e Claude li usa come manuale quando lavora.

DOCUMENTO	COSA CONTIENE
SDD framework + Mega Prompt	Il metodo "spec prima del codice" per intero, più il Mega Prompt: la versione leggera per i progetti piccoli — 4 comandi (<code>/plan</code> → <code>/build</code> → <code>/verify</code> → <code>/ship</code>) al posto di un framework completo.
Agent design principles	Le regole per progettare agent fatti bene: un agent = un lavoro, input e output chiari, mai fidarsi senza verifica.
Tools setup	La guida agli strumenti esterni che gli agent possono usare e come collegarli.
Template CLAUDE.md di progetto	Il file di regole pronto da copiare in ogni progetto nuovo: dice a Claude come comportarsi <i>in quel progetto specifico</i> (l'orchestrator decide per tutto il computer, questo per la singola cartella).

Tutti i comandi in una tabella

COMANDO / SKILL	COSA FA
<code>/stack-bootstrap</code>	Monta tutto lo stack: orchestrator, vault, plugin, skill, comandi.
<code>/stack-verify</code>	Controlla che ogni pezzo dello stack sia al suo posto.
<code>/agent-spec</code>	Scriva il contratto del nuovo agent (cosa entra, cosa esce, regole).
<code>/agent-tasks</code>	Trasforma la spec nella lista dei lavori.
<code>/agent-execute</code>	Esegue i lavori uno alla volta, con salvataggio a ogni passo.
<code>/agent-verify</code>	Controlla che la spec sia coperta al 100%.
<code>/agent-ship</code>	Consegna l'agent finito.
<code>/graphify .</code>	Costruisce il grafo della cartella in cui sei — concetti e collegamenti.
<code>create-agent</code>	"Fammi un agent che..." → porta la richiesta dentro la agent factory.
<code>vault-ingest</code>	Mette materiale nuovo nel vault, ordinato e indicizzato.
<code>vault-query</code>	Cerca e risponde dal vault.
<code>vault-lint</code>	Pulizia e salute del vault.
<code>deploy-project</code>	Mette online un progetto: audit sicurezza + test + pubblicazione.
<code>shinen-design</code>	Applica la grafica SHIN-EN a tool e dashboard.
<code>handoff</code>	Riassunto di fine sessione per ripartire senza rispiegare.

Da qui in poi

Hai la mappa. Nelle lezioni vedi ogni pezzo lavorare dal vivo: la agent factory che monta il primo agent, le vault skills che riempiono la libreria, SHIN-EN che impagina i PDF. **Non devi memorizzare niente:** torna a questa guida quando ti serve, e ricorda la regola che vale per tutto — qualsiasi dubbio, chiedi a Claude.

Repo + README: github.com/Lucioway/way-stack